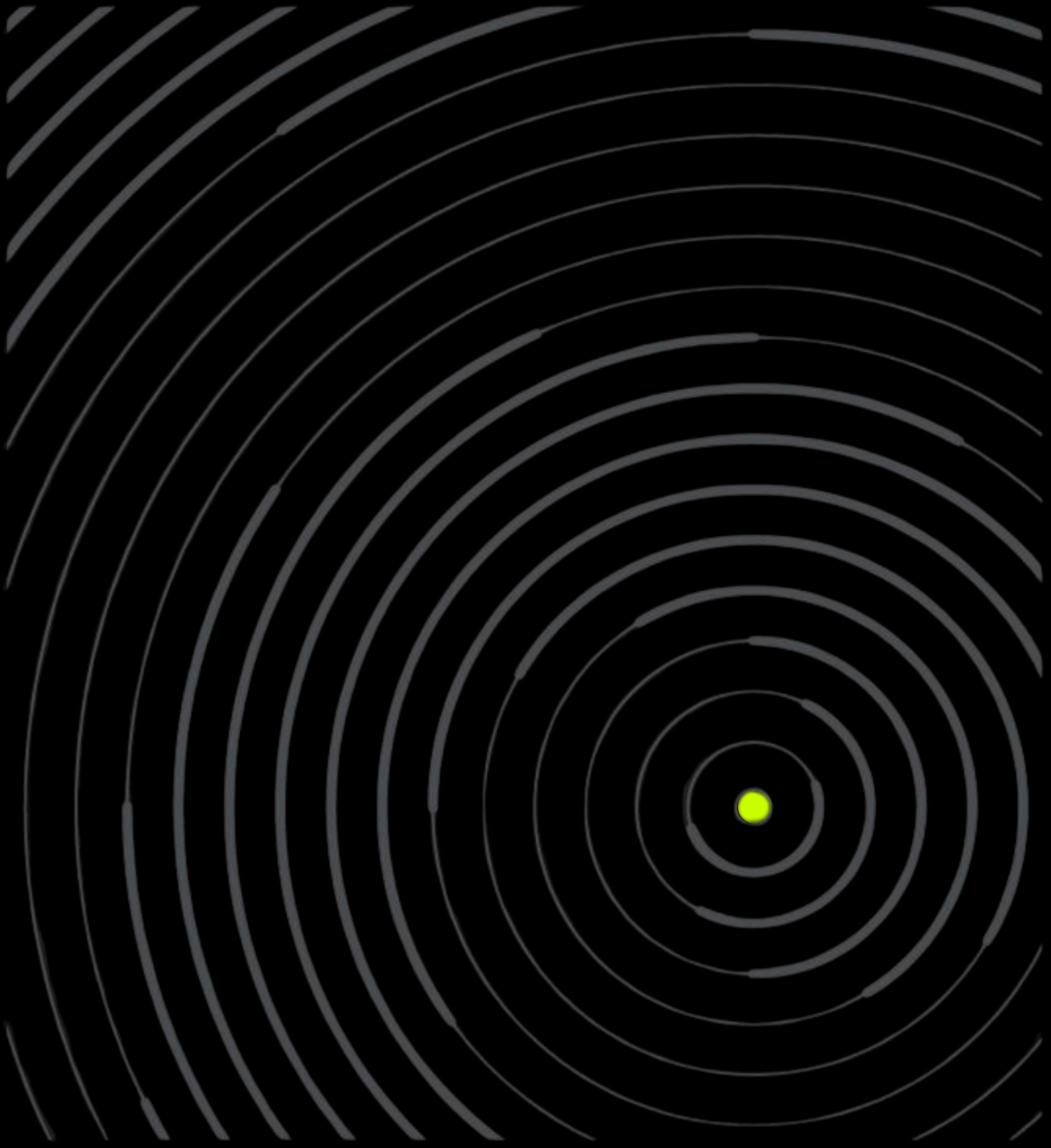
OxTeam.

2026



SECURITY ASSESSMENT

PREPARED FOR

MY PET HOOLIGAN

Table of Contents

Revision History & Version Control

Version	Date	Author's	Description
1.0	January 19, 2026	Gurkirat, Manoj	Initial Report

0xTeam conducted a comprehensive Security Audit on MPH Game Marketplace to ensure the overall code quality, security, and correctness. The review focused on ensuring that the code functions as intended, identifying potential vulnerabilities, and safeguarding the integrity of MPH Game Marketplace's operations against possible attacks.

1.0 Executive Summary

1.1 Overview

0xTeam partnered with the MPH Game Marketplace team to conduct a security audit of their smart contracts. The audit involved a manual review of the codebase to ensure the contracts are secure and reliable before deployment. This process was carried out to provide confidence to the project team and its community regarding the safety of the deployed contracts.

1.2 Scope

The scope of this audit involved a thorough analysis of the MPH Game Marketplace Smart contracts, focusing on evaluating its quality, rigorously assessing its security, and carefully verifying the correctness of the code to ensure it functions as intended without any vulnerabilities.

Language	Date of Received Scope
SOLIDITY	Jan 16, 2026

Initial Commit	Fixed Commit
c88d2a40f70d964edc5067ba507c9aee5cceb81d	N/A

Repository

https://github.com/TheZuckaNator/vercel-store/commits/main/contracts

In-Scope Files

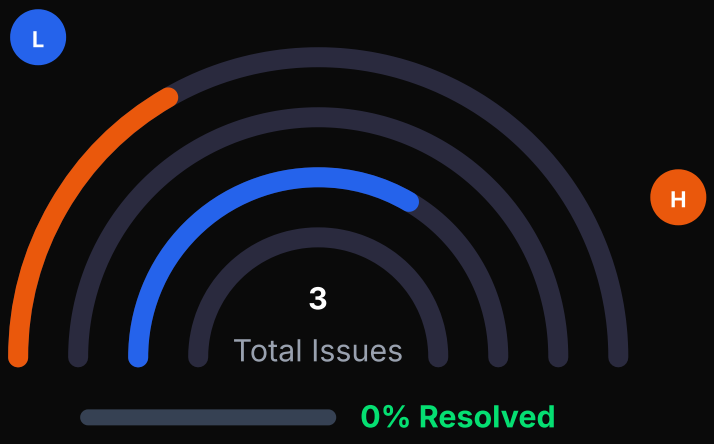
- Contracts/MPHGameMarketplace1155.sol
- Contracts/MPHGameMarketplaceNative.sol

- **OUT-OF-SCOPE:** External Smart contracts code, other imported code.

1.3 Vulnerability Summary

1
High

0
Resolved



High 1 Medium 0 Low 2 Info 0

1.4 Issue Status Summary

Status / Severity	High	Medium	Low	Info
Open	1	0	2	0
Acknowledged	0	0	0	0
Partially Resolved	0	0	0	0
Resolved	0	0	0	0

Open	Acknowledged	Partially Resolved	Resolved
3	0	0	0
100%	0%	0%	0%

1.5 Type of Severity

High

Issues that pose an immediate and severe risk to the protocol's funds, user assets, or core system integrity, or significant security vulnerabilities that could lead to substantial financial loss or compromise of important protocol functionality. These issues require prompt attention and immediate remediation before deployment to prevent potential exploitation or complete loss of funds.

Medium

Moderate security concerns that could potentially be exploited under specific conditions or may impact protocol functionality. While not immediately critical, these issues should be resolved to ensure long-term security and reliability of the system.

Low

Minor issues that do not pose significant security risks but represent deviations from best practices or code quality standards. These findings improve code maintainability, readability, and adherence to industry conventions.

Informational

Observations and recommendations that enhance code quality, gas efficiency, or overall design. These are non-security findings that provide suggestions for optimization, documentation improvements, or architectural considerations.

1.6 Type of Issue Status

Open Finding awaiting remediation by the development team.	**Resolved** Issue has been addressed and verified by the audit team.
Acknowledged Team has reviewed and accepted the risk without immediate fix.	**Partially Resolved** Mitigation applied but complete resolution pending further work.

2. Techniques and Methods

The following techniques, methods, and tools were used to conduct a comprehensive security review of the smart contracts.

Structural Analysis

Examination of overall design, architecture, and code organization to ensure scalability, maintainability, and adherence to industry standards.

Static Analysis

Automated scanning for common vulnerabilities including reentrancy, arithmetic errors, access control issues, and potential denial-of-service vectors.

Manual Code Review

Line-by-line examination of contract logic to verify intended functionality, validate business requirements, and identify issues beyond automated detection.

Dynamic Analysis

Execution of contracts in controlled environments with comprehensive test cases to observe behavior, verify gas efficiency, and ensure correct state transitions.

Platforms and Tools Used to Audit

Remix IDE | Foundry | Slither | Hardhat | Solhint

3. Summary of Findings

ID	Title	Severity	Status
H-01	Missing ETH Withdrawal Mechanism Causes Permanent Fund Lock	High	Open
L-01	Missing Zero-Address Validation	Low	Open
L-02	State Variable Could Be Immutable	Low	Open

● Resolved ● Partially Resolved ● Acknowledged ● Open

4. Findings

[H-1] Missing ETH Withdrawal Mechanism Causes Permanent Fund Lock

HIGH Open

CONTRACT	LOCATION
MPHGameMarketplaceNative	N/A

Description

The MPHGameMarketplaceNative contract is capable of receiving ETH both via its explicit receive() function:

```
receive() external payable {}
```

and as part of marketplace execution flows through the buyNFT and buyMultipleNFTs functions.

When buyers call buyNFT or buyMultipleNFTs, they are required to send totalPrice + fee in ETH. During execution, the contract transfers ETH to the seller and the designated feeReceiver. However, during fee accounting, an additional fee amount remains in the contract balance after each successful trade.

As a result, ETH accumulates in the marketplace contract during normal trading activity. The contract does not provide any public or admin-accessible mechanism to withdraw ETH held in address(this).balance. Consequently, any ETH accumulated as fees, as well as ETH sent directly to the contract address, becomes permanently unrecoverable.

Impact

- ETH fees accumulate in the contract during normal marketplace operations and accumulated ETH cannot be withdrawn by protocol administrators.
- ETH sent directly to the contract address (e.g., wallet transfers) is also permanently locked

The PoC demonstrates a normal marketplace trade where:

1. The buyer sends totalPrice + fee ETH as required by the contract
2. The seller correctly receives totalPrice - fee
3. The feeReceiver correctly receives fee
4. **An additional fee amount remains in the marketplace contract balance**
5. There is no mechanism to withdraw this ETH

The screenshots below show that:

- Seller and fee receiver balances increase as expected
- The marketplace contract balance increases by fee
- The ETH remains permanently locked after trade execution

```
it("POC: Admin cannot withdraw funds received as fee from the trading in the marketplace", async function () {
  const tokenId = 1;
  const amount = 1;
  const price = ethers.parseEther("100"); // 0.1 ETH per item
  const deadline = (await time.latest()) + 3600;
  const signature = await createSignature(seller, nftAddress, tokenId, amount, price, 0, deadline);

  const sellerBalanceBefore = await ethers.provider.getBalance(seller.address);
  console.log("Seller ETH before:", ethers.formatEther(sellerBalanceBefore));

  const buyerETHBalanceBefore = await ethers.provider.getBalance(buyer.address);
  console.log("Buyer ETH before:", ethers.formatEther(buyerETHBalanceBefore));

  const feeRecieverBalanceBefore = await ethers.provider.getBalance(feeReceiver.address);
  console.log("Fee Receiver ETH before:", ethers.formatEther(feeRecieverBalanceBefore));

  console.log("Balance of the contract before ",ethers.formatEther(await  ethers.provider.getBalance(marketplaceAddress)));

  const buyerNFTBefore = await nft.balanceOf(buyer.address, tokenId);

  const totalPrice = price * BigInt(amount);
  const fee = (totalPrice * 25n) / 1000n; // 2.5%

  console.log("Total Price of the NFT's:", ethers.formatEther(totalPrice));
  console.log("Total Fee:", ethers.formatEther(fee));

  await expect(
    marketplace.connect(buyer).buyNFT(
      nftAddress, tokenId, amount, price, deadline, seller.address, signature,
      { value: totalPrice + fee }
    )
  ).to.emit(marketplace, "NFTBought")
    .withArgs(nftAddress, tokenId, buyer.address, seller.address, amount, totalPrice);

  // Check NFT transferred
  expect(await nft.balanceOf(buyer.address, tokenId)).to.equal(buyerNFTBefore + BigInt(amount));
  expect(await nft.balanceOf(seller.address, tokenId)).to.equal(5 - amount);

  // Check seller received ETH (minus fee)
  const sellerBalanceAfter = await ethers.provider.getBalance(seller.address);
  console.log("Seller Total ETH after:", ethers.formatEther(sellerBalanceAfter));
  console.log("Seller recieved ETH after", ethers.formatEther(sellerBalanceAfter - sellerBalanceBefore));

  const feeReceiverAfter = await ethers.provider.getBalance(feeReceiver.address);
  console.log("Fee Receiver ETH after:", ethers.formatEther(feeReceiverAfter));

  const buyerETHBalanceAfter = await ethers.provider.getBalance(buyer.address);
  console.log("Buyer Total ETH after:", ethers.formatEther(buyerETHBalanceAfter));
  console.log("Buyer spent ETH after:", ethers.formatEther(buyerETHBalanceBefore - buyerETHBalanceAfter));
  console.log("Balance of the contract after ",ethers.formatEther(await  ethers.provider.getBalance(marketplaceAddress)));
});
```

```
gurkiratsingh@Gurkirats-MacBook-Air MPH Marketplace % npx hardhat test test/MPHGameMarketplaceNative.test.cjs


  MPHGameMarketplaceNative
    Unit Tests
Seller ETH before: 9999.999668931524792088
Buyer ETH before: 10000.0
Fee Receiver ETH before: 10000.0
Balance of the contract before  0.0
Total Price of the NFT's: 100.0
Total Fee: 2.5
Seller Total ETH after: 10097.499668931524792088
Seller recieved ETH after 97.5
Fee Receiver ETH after: 10002.5
Buyer Total ETH after: 9897.499834753285105585
Buyer spent ETH after: 102.500165246714894415
Balance of the contract after  2.5
      ✔ POC: Admin cannot withdraw funds received as fee from the trading in the marketplace


  1 passing (598ms)
```

There is no way to withdraw this fee from the contract

Recommendation

There could be two solutions for this:

- **Add an Admin-Restricted ETH Withdrawal Mechanism**
  Implement an admin-only sweep function that allows protocol administrators to withdraw any ETH held in the contract. This ensures that ETH accumulated due to accounting errors, misconfiguration, or direct transfers can be recovered and prevents permanent fund lock.
- **Enforce a Single, Consistent Fee Model**
  The protocol should adopt a single fee model (either buyer-paid or seller-paid) and ensure that ETH accounting is consistent with that model because in the MPHGameMarketplace1155 the fee is only charged from the buyer. All ETH received as fee should be forwarded to the fee receiver, leaving no residual ETH in the contract after successful trades.

[L-1] Missing Zero-Address Validation

LOW Open

CONTRACT	LOCATION
MPHGameMarketplace1155	constructor(),setMarketPlace(), setVerifier()

Description

Several functions across the MPHGameMarketplace1155 contract, including the constructor of MPHGameMarketplace1155, assign address-type parameters without validating that the provided address is non-zero.

Impact

If address parameters are not validated, critical roles or dependencies may be set to address(0), which might cause contract functionality to be partially or fully inoperable.

Recommendation

Add zero-address checks in the specified locations.

[L-2] State Variable Could Be Immutable

LOW Open

CONTRACT	LOCATION
MPHGameMarketplaceNative	paymentToken

Description

paymentToken is only assigned in the constructor. Therefore it can be made immutable as immutable values are cheaper to read. State variables that are not updated following deployment should be declared immutable to save gas.

Impact

While this issue doesn't directly impact the functionality of the contract, the contract can benefit from the use of constant and immutable keywords for variables that do not change after deployment. This could save gas costs by storing the variables directly in the bytecode.

Recommendation

Add the immutable attribute to state variables that never change or are set only in the constructor.

```
IERC20 public immutable paymentToken;
```

Disclaimer

The 0xTeam smart contract audit is not a security warranty, investment advice, or an endorsement of the MPH Game Marketplace platform. This audit does not guarantee that the code is completely free of vulnerabilities, as security risks can evolve over time.

Users and investors should conduct their own due diligence before interacting with the protocol. 0xTeam is not responsible for any financial losses, hacks, or exploits that may occur after the audit. The findings in this report reflect the security state of the code at the time of assessment.

By using this report, you acknowledge that smart contract security is an ongoing process and that 0xTeam shall not be held liable for any direct or indirect damages resulting from the use of the audited code.

About 0xTeam

0xTeam is a Web3-focused cybersecurity company dedicated to strengthening blockchain systems and decentralized applications. We help teams adopt blockchain technology with confidence by ensuring their smart contracts and infrastructure are secure, reliable, and production-ready.

Security is at the core of everything we do. Our services are designed to support smooth adoption across the Web3 ecosystem, enabling projects to scale safely while minimizing technical and financial risk.

0xTeam continues to grow its partnerships with developers, startups, and organizations across Web3, working together to build secure, resilient, and future-ready decentralized products.

Website: https://0xteam.space

Contact: info@0xteam.space

0xTeam.